

Python for DSA

Day 1: Lists & Tuples

Goal: Understand core data structures

What is a List?

Think of a list like a **shopping bag** 🛒

It allows you to store multiple items together in a single variable.

Example:

```
shopping = ["milk", "bread", "eggs"]
```



List Properties



Ordered

The position of each element matters. The order you insert is preserved.



Mutable

You can freely change, add, or remove elements after the list is created.



Duplicates

Lists can contain multiple identical values without any issues.



Any Data Type

You can mix integers, strings, objects, and even other lists inside.

Accessing Elements

Index = the numerical position of the element inside the list.

First Item

In Python, indexing starts at 0.

```
shopping[0] # Returns first item
```

Last Item

Negative indexing starts from the end.

```
shopping[-1] # Returns last item
```


Add Elements

Append (End)

Adds a single element to the very end of the list.

```
fruits.append("grapes")
```

Insert (Specific)

Places an element at a specific index position.

```
fruits.insert(1, "kiwi")
```


Remove Elements

Remove by Value

Finds and removes the first occurrence of the specific value.

```
fruits.remove("banana")
```

Pop by Index

Removes and returns an item at an index (defaults to the last).

```
fruits.pop() # last item  
fruits.pop(0) # first item
```


Useful Operations



Find Length

```
len(fruits) # Returns item count
```



Check Existence

```
"mango" in fruits # Returns True or False
```



Find Exact Index

```
fruits.index("mango") # Returns index number
```


Sorting

Ascending Order

Sorts the list in natural (A-Z or 0-9) order in-place.

```
fruits.sort()
```

Descending Order

Use the reverse parameter to sort backwards.

```
fruits.sort(reverse=True)
```


Slicing (Very Important!)

Syntax	Meaning	Example Action
<code>nums[1:3]</code>	From index 1 up to (but not including) index 3	Gets elements at index 1 and 2
<code>nums[:3]</code>	From the start (index 0) up to index 3	Gets the first 3 elements
<code>nums[2:]</code>	From index 2 all the way to the end	Skips the first 2 elements
<code>nums[::-1]</code>	Steps backwards through the list	Reverses the entire list

Real-Life Example

Understanding concepts through real-world analogies makes DSA highly intuitive.

- **Train seats** represent the **List**
- **Seat number** represents the **Index**
- **Passengers boarding/leaving** represent **insert()** and **remove()**



What is a Tuple?

Think of a Tuple like an **Aadhaar Card** 🪪

Once it is created, the information inside it **cannot be changed**.

Example:

```
aadhaar = ("Saurav", "Indore", 1234567890)
```



Tuple Key Points



Immutable

Absolutely no changes can be made once the tuple is instantiated.



Faster

Because their size is fixed, Python iterates through them faster than lists.



Fixed Data

Perfect for storing constant configurations or coordinates.

Tuple Use Case

Multiple Values Return

Tuples are frequently used to gracefully return multiple values from a single function call without needing complex objects.

Function Example

```
def min_max(nums): return  
(min(nums), max(nums))
```

List Comprehension

A short, readable, and incredibly powerful way to create lists mathematically in Python.

```
[i*i for i in range(1, 6)]
```

Output: [1, 4, 9, 16, 25]

With Condition

Filtering Data

List comprehensions allow you to filter elements easily using an **if** condition, drastically reducing the lines of code required.

Syntax

```
[x for x in range(1,11) if x % 2 == 0]
```

Output: [2, 4, 6, 8, 10]

Practice Problems



Problem 1

Write a function to efficiently **reverse a list**.



Problem 2

Given a list of integers, **find all even numbers**.



Problem 3

Take two separate lists, **merge them, and sort** the result.



Summary

Concept	Key Characteristic	Primary Use Case
List	Mutable (Changeable)	General data storage & dynamic collections
Tuple	Immutable (Unchangeable)	Fixed configurations & multiple value returns
Slicing	Subset extraction	Highly important tool for DSA array manipulation
List Comprehension	Powerful shortcut	Concise one-line list creation and filtering



Homework

Solve all 3 practice problems.

Be ready to explain your logic in the next session.



Next Topic: Dictionaries & Sets

Image Sources



<https://static.vecteezy.com/system/resources/thumbnails/071/766/301/small/a-brown-canvas-tote-bag-filled-with-fresh-groceries-including-asparagus-eggs-bread-and-fruit-isolated-on-white-background-free-png.png>

Source: www.vecteezy.com



https://st4.depositphotos.com/9006966/39650/i/1600/depositphotos_396508892-stock-photo-passenger-train-interior-empty-seats.jpg

Source: depositphotos.com



https://static.vecteezy.com/system/resources/previews/020/143/873/non_2x/modern-and-clean-business-id-card-template-professional-id-card-design-template-vector.jpg

Source: www.vecteezy.com